



UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

**COMPARAÇÃO DE ALGORITMOS DE ORDENAÇÃO: BINARY
INSERTION SORT, INSERTION SORT E MERGE SORT**

Giulia Ferrari – RA – G783HF2

Joaquim José Mendes de Souza – RA – N079657

Lucas Inocêncio de França - RA – N239239

Maria Luiza Alves Dominicali – RA – T296DC0

Pedro Eduardo Silva Bigossi – RA – N296445

Rodrigo Pamplona Soares – RA – N2409H8

Washington Santos Souza Filho – RA – N046FD9

São José dos Campos, 25 de agosto de 2024.



UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

COMPARAÇÃO DE ALGORITMOS DE ORDENAÇÃO: BINARY
INSERTION SORT, INSERTION SORT E MERGE SORT

Atividades Práticas Supervisionadas do
3º e 4º Semestres do Curso de Ciência
da Computação da **Universidade
Paulista – UNIP.**

Coordenador: Prof. Fernando A. **Gotti**

São José dos Campos, 25 de agosto de 2024.



UNIP – UNIVERSIDADE PAULISTA

Curso de Ciência da Computação

FICHA DE APROVAÇÃO

TEMA:

COMPARAÇÃO DE ALGORITMOS DE ORDENAÇÃO: BINARY INSERTION SORT, INSERTION SORT E MERGE SORT

Este Trabalho foi aprovado como avaliação semestral da disciplina Atividades Práticas Supervisionadas - APS

Os alunos receberam as seguintes notas:

Aluno	Trabalho Impresso	Apresentação	Média (Total)
Giulia			
Joaquim			
Lucas			
Maria			
Pedro			
Rodrigo			
Washington			

Professor Orientador

Prof. Stefano B. B. R. P. Mathias

São José dos Campos, ____ de _____ de 2024.

RESUMO

Organizar grandes volumes de dados é uma tarefa fundamental nos dias atuais. Para facilitar essa organização, foram desenvolvidos algoritmos de ordenação, que são estruturas lógicas projetadas para organizar dados de maneira eficiente. Neste estudo, implementamos e analisamos três algoritmos de ordenação distintos: o *Binary Insertion Sort*, o *Insertion Sort* e o *Merge Sort*. Nosso objetivo principal foi avaliar as vantagens e desvantagens de cada um e realizar uma comparação de desempenho. Para isso, foram utilizados conjuntos de dados de tamanhos distintos. Esta análise permitiu identificar as situações em que cada algoritmo é mais eficiente, proporcionando uma visão sobre a aplicação prática de cada método em diferentes situações.

Palavras-chave: Algoritmos de ordenação, *InsertionSort*, *MergeSort*, *BinaryInsertionSort*.

ABSTRACT

Organizing large volumes of data is a fundamental task nowadays. To facilitate this organization, sorting algorithms were developed, which are logical structures designed to organize data efficiently. In this study, we implemented and analyzed three different sorting algorithms: Binary InsertionSort, InsertionSort and MergeSort. Our main objective was to evaluate the advantages and disadvantages of each one and carry out a performance comparison. For this, data sets of different sizes were used. This analysis made it possible to identify the situations in which each algorithm is most efficient, providing insight into the practical application of each method in different situations.

Key words: Sorting algorithms, *InsertionSort*, *MergeSort*, *BinaryInsertionSort*.

LISTA DE FIGURAS

Figura 1 - Análise assintótica.....	5
Figura 2 – Big O Notation	6
Figura 3 - Insertion Sort.....	7
Figura 4 - Busca binaria	8
Figura 5 - Exemplo de Ordenação com Merge Sort	10
Figura 6 - Função Entrada()	12
Figura 7 - Função MedicaoTempo()	12
Figura 8 – Função Binary Insertion Sort	13
Figura 9 - Função BuscaBinaria()	14
Figura 10 – Função InsertionSort()	15
Figura 11 - Função MergeSort()	16
Figura 12 - Saída do terminal Merge Sort	18
Figura 13 - Saída do terminal Insertion Sort	19
Figura 14 - Saída do terminal Binary Insertion Sort.....	20

LISTA DE TABELAS

Tabela 1 - Resultados do Merge Sort.....	17
Tabela 2 - Resultados Insertion Sort	18
Tabela 3 - Resultados Binary Insertion Sort	20
Tabela 4 - Comparação entre os algoritmos.....	21

SUMÁRIO

1 - INTRODUÇÃO.....	1
2 - REFERENCIAL TEÓRICO	3
2.1 - ALGORITMOS DE ORDENAÇÃO	3
2.2 - COMPLEXIDADE DE ALGORITMOS.....	3
2.2.1 - ANÁLISE ASSINTÓTICA	4
2.3 - INSERTION SORT	6
2.3.1 - FUNCIONAMENTO.....	6
2.3.2 - COMPLEXIDADE COMPUTACIONAL.....	7
2.3.3 - VANTAGENS E DESVANTAGENS	7
2.4 - BINARY INSERTION SORT	8
2.4.1 - FUNCIONAMENTO.....	8
2.4.2 - COMPLEXIDADE COMPUTACIONAL.....	9
2.4.3 - VANTAGENS E DESVANTAGENS	9
2.5 - MERGE SORT.....	9
2.5.1 - FUNCIONAMENTO.....	9
2.5.2 - COMPLEXIDADE COMPUTACIONAL.....	10
2.5.3 - VANTAGENS E DESVANTAGENS	10
3 - DESENVOLVIMENTO.....	12
3.1 - BINARY INSERTION SORT	13
3.2 - INSERTION SORT	14
3.3 - MERGE SORT.....	15
5 - RESULTADOS E DISCUSSÃO.....	17
5.1 - RESULTADOS DO MERGE SORT	17
5.2 - RESULTADOS DO INSERTION SORT.....	18
5.3 - RESULTADOS DO BINARY INSERTION SORT	20
5.4 - COMPARAÇÃO DE RESULTADOS ENTRE ALGORITMOS.....	21
7 - CONSIDERAÇÕES FINAIS	22
REFERÊNCIAS BIBLIOGRÁFICAS	22

1 - INTRODUÇÃO

A ordenação de dados é um processo fundamental no mundo da tecnologia, sendo essencial para a organização de grande volume de dados em vários contextos, desde a otimização de buscas em sistemas de grandes dados ou até desempenho em todo tipo de aplicação de grande escala. Os algoritmos de ordenação têm um papel crucial ao organizar dados facilitando a manipulação e a análise dos mesmos, seja em pequenas listas ou em grandes volumes de dados. Para atender a esses diversos cenários, existem diversos algoritmos de ordenação, onde cada um possui suas características específicas que afetam de cada modo a sua eficiência e aplicabilidade em diferentes contextos. Este trabalho focará em análise comparativa dos algoritmos *Insertion Sort*, *Merge Sort* e *Binary Insertion Sort*, avaliando suas particularidades, vantagens e limitações (LOPES, 2023).

O *Insertion Sort* é um algoritmo de ordenação que se destaca por sua simplicidade e eficiência em listas pequenas ou quase já ordenadas. Ele funciona construindo a sequência ordenada pouco a pouco, e inserindo cada novo elemento em sua posição correta dentro do subconjunto principal já ordenado. Este método tem uma complexidade de tempo $O(n^2)$, que significa que seu desempenho pode ser lento para listas grandes, mas em listas pequenas ou já quase ordenadas, sua execução é extremamente rápida, tornando-o útil para cenários específicos ou em sistemas onde a ordenação é aplicada em subconjuntos menores de dados. Além disso, o *Insertion Sort* é um algoritmo estável, preservando a ordem relativa dos elementos iguais, uma característica importante para dados com múltiplos atributos de ordenação. (GEEKSFORGEEKS, 2024).

O *Merge Sort*, em sua vez, é mais recomendado para grandes listas e desordenadas, pois utiliza a técnica de "dividir para conquistar" para otimizar a ordenação. Esse algoritmo divide a lista em sublistas menores, ordena cada uma e, em seguida, as mescla em uma única lista e ordenada. Com uma complexidade de tempo $O(n \log n)$, o *Merge Sort* é eficiente mesmo com grandes volumes de dados e é utilizado em contextos onde a estabilidade da ordenação e a capacidade de lidar com grandes volumes de informação são essenciais (W3SCHOOLS, 2024). No entanto, um ponto a considerar é o uso adicional de memória de processamento (Memória RAM), uma vez que ele precisa de espaço extra para armazenar as sublistas durante o processo

de mesclagem, o que pode ser uma limitação em sistemas com restrições de memória (RIZZON, 2023).

Por último, o *Binary Insertion Sort* é uma variação aprimorada do *Insertion Sort*, que utiliza a técnica de busca binária para localizar a posição correta de inserção de cada elemento. Ao invés de uma busca linear, ele reduz o número de comparações no uso da busca binária, o que, talvez, possa melhorar significativamente o desempenho em alguns casos. Embora o *Binary Insertion Sort* compartilhe a mesma complexidade de tempo $O(n^2)$ no pior caso, ele apresenta vantagens em listas parcialmente ordenadas, onde a busca binária acelera a localização do ponto de inserção, tornando-o mais eficiente que o *Insertion Sort* (SEEDGEWICK; WAYNE, 2016).

Dessa forma, o trabalho busca explorar o funcionamento desses três algoritmos, analisando seus desempenhos e particularidades por meio de implementações usando a linguagem de programação *Python* e testes em dados de diferentes tamanhos. Esta análise comparativa permitirá avaliar as vantagens e limitações de cada algoritmo, possibilitando a compreensão profunda sobre sua aplicabilidade e eficiência em diversos cenários da ordenação de dados.

2 - REFERENCIAL TEÓRICO

Este tópico está estruturado da seguinte maneira, primeiro apresentamos uma breve explicação sobre algoritmos de ordenação, em sequência abordaremos a complexidade de algoritmos e por fim explicaremos os conceitos e funcionalidades dos algoritmos selecionados.

2.1 - ALGORITMOS DE ORDENAÇÃO

Os algoritmos de ordenação são procedimentos utilizados para organizar os elementos de um conjunto em uma ordem específica, geralmente crescente ou decrescente. Esses algoritmos são fundamentais, pois otimizam a organização dos dados, facilitando o acesso e a manipulação eficiente das informações. (CORMEN; ET AL, 2012)

A ordenação é comum em muitas aplicações, como, por exemplo, a organização de dados em sistemas de bancos de dados. Cada algoritmo possui características próprias, vantagens e desvantagens, variando em termos de complexidade de tempo e espaço, estabilidade e eficiência. (CORMEN; ET AL, 2012)

2.2 - COMPLEXIDADE DE ALGORITMOS

A preocupação com a complexidade de algoritmos é fundamental para projetar algoritmos eficientes. Embora seja possível desenvolver um algoritmo e, em seguida, analisar sua complexidade para verificar sua eficiência, o ideal é considerar a eficiência desde a concepção do algoritmo. (RIBEIRO, 2023)

Para comparar dois algoritmos que resolvem o mesmo problema, é necessário definir medidas que expressem sua eficiência. Costuma-se avaliar um algoritmo em termos de tempo de execução, espaço de memória de armazenamento temporário e a quantidade de instruções no código. (RIBEIRO, 2023)

O tempo de execução refere-se ao período necessário para que o algoritmo seja executado. Podemos considerar o tempo absoluto em minutos ou segundos, essa métrica depende da máquina, assim sendo menos interessante. O espaço de memória de armazenamento temporário refere-se à quantidade de memória necessária para a execução do algoritmo. A quantidade de comandos seria outra métrica, onde,

independentemente do tamanho da entrada "N", o código executa a mesma quantidade de comandos, representada pelo Big O como $O(1)$. (RIBEIRO, 2023)

A complexidade temporal de um algoritmo é comumente representada pela notação Big O. Essa notação descreve como o tempo de execução cresce em função do tamanho da entrada. (RIBEIRO, 2023)

Por exemplo, um algoritmo de busca linear possui uma complexidade de $O(n)$, onde "n" representa o tamanho da entrada. Isso indica que, conforme a entrada aumenta, o tempo de execução do algoritmo também cresce linearmente. Existem algoritmos mais eficientes, como o *Merge Sort*, que apresentam um tempo de execução de $O(n \log n)$, considerado mais rápido do que $O(n)$. (RIBEIRO, 2023)

Em casos onde temos um loop aninhado, a complexidade temporal pode resultar em $O(n^2)$, onde "n" é o tamanho da entrada. Isso ocorre porque, para cada elemento na primeira iteração do loop externo, o loop interno é executado "n" vezes, resultando em um crescimento quadrático do tempo de execução. (RIBEIRO, 2023)

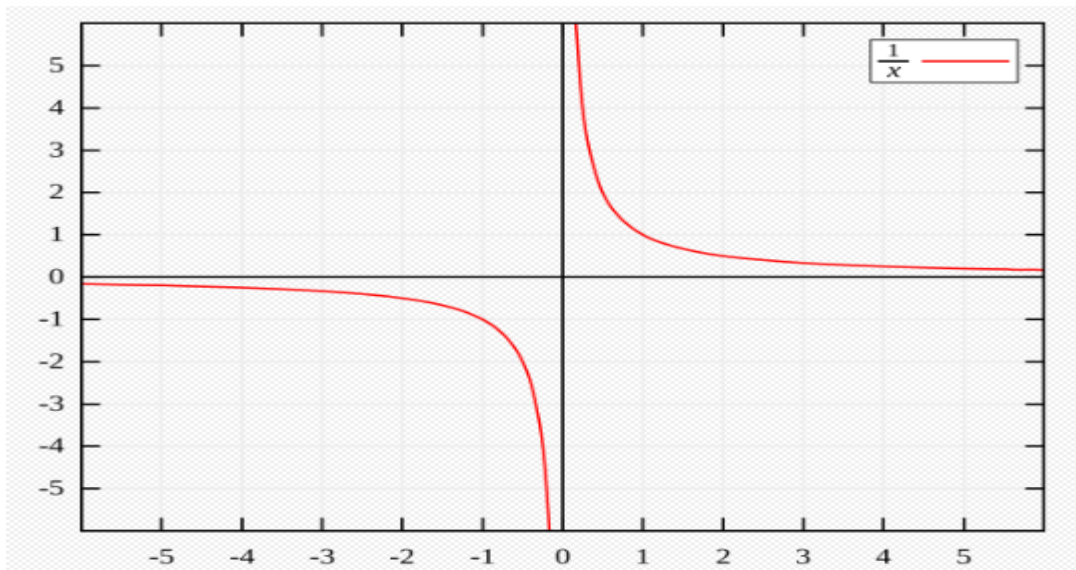
2.2.1 - ANÁLISE ASSINTÓTICA

A análise assintótica é um método que permite analisar o tempo de execução de um algoritmo, ou seja, sua taxa de crescimento, e está relacionada à complexidade de algoritmos. (BRUNET, 2019)

O termo "assintótica" deriva da palavra "assíntota". Em uma curva, uma assíntota é uma linha que representa uma distância infinita em relação a um ponto P, conforme esse ponto se afasta em direção ao infinito, sem nunca realmente se encontrar com a linha. (SANCHO, 2023)

Segundo Araujo (2023), a análise assintótica é conforme visto na Figura 1.

Figura 1 - Análise assintótica



Fonte: Araujo (2023)

De maneira simples, imagine que essa linha vermelha continua se aproximando do valor zero, sem jamais chegar ao ponto zero, se estendendo indefinidamente. (BRUNET, 2019)

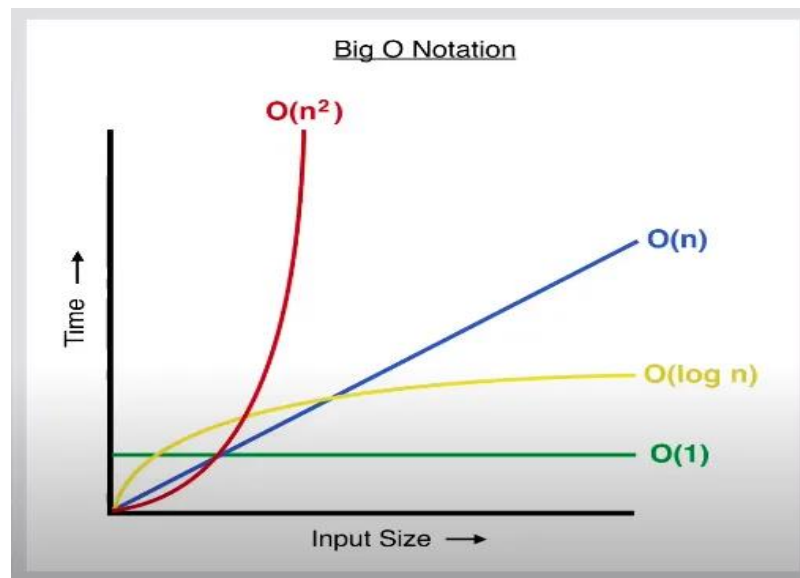
Quando lidamos com entradas extremamente grandes ou quando “n” tende ao infinito, utilizamos a análise assintótica. (BRUNET, 2019)

Além de medir o tempo de execução, também podemos analisar os extremos ou limites dos algoritmos. No pior caso (*Big O*), consideramos o maior tempo de execução possível, representando o cenário mais desfavorável. Esse cálculo é baseado no tempo máximo necessário para processar todas as entradas de tamanho N. No caso médio (*Theta* Θ), definimos o tempo limite como o tempo de execução esperado, calculando a média para todas as entradas. No melhor caso (*Ômega* Ω), o limite é o menor tempo de execução ou o resultado mais favorável que podemos alcançar. (SONG, [s.d.])

O "pior caso" é o cenário mais frequentemente analisado. (SANCHO, 2023)

A Figura 2 apresenta algumas complexidades de *Big O*:

Figura 2 – Big O Notation



Fonte: Kielkowski (2020)

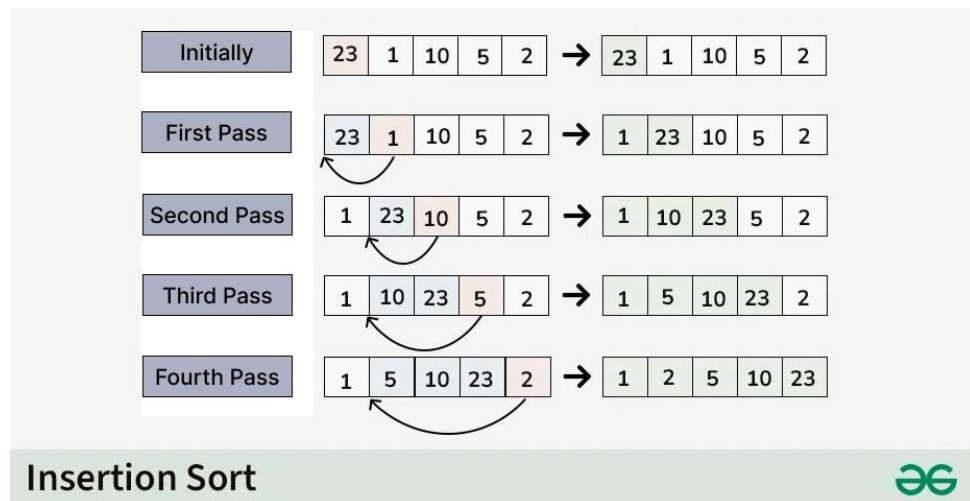
2.3 - INSERTION SORT

A Ordenação por Inserção é um algoritmo de ordenação que organiza listas desordenadas ao inserir cada elemento em uma posição correta dentro de uma seção já ordenada. Ele é estável, ou seja, mantém a ordem relativa dos elementos iguais. O funcionamento do algoritmo começa com o segundo elemento, que é comparado ao primeiro; se necessário, os dois são trocados. O terceiro elemento é então comparado com os anteriores e posicionado adequadamente. Esse processo se repete até que toda a lista esteja ordenada. O *Insertion Sort* é um método in-place, pois não requer espaço adicional além do necessário para a lista original. (GEEKSFORGEES, 2024)

2.3.1 - FUNCIONAMENTO

Para ordenar um *array*, com os valores = [8, 0, 3, 4, 9, 6], o algoritmo considera os dois primeiros elementos, [8, 0]. Após aplicar o *Insertion Sort*, obtém-se [0, 8]. Depois, ao incluir o terceiro elemento, forma-se [0, 8, 3], que se transforma em [0, 3, 8]. Este processo continua até que o *array* completo esteja ordenado, a Figura 3 ilustra o funcionamento do *Insertion Sort*. (BRUNET, 2019)

Figura 3 - Insertion Sort



Fonte: GeekForGeeks (2024)

A estabilidade do algoritmo garante que a ordem de elementos iguais no *array* original seja preservada. A Ordenação por Inserção é *in-place*, rearranjando elementos no próprio *array*. Embora não seja eficiente para grandes entradas, o *Insertion Sort* se destaca como um dos algoritmos mais rápidos para listas muito pequenas, frequentemente superando o *Quick Sort* em determinadas situações, onde boas implementações do *Quick Sort* utilizam o *Insertion Sort* para *array* menores que um certo limite. (BRUNET, 2019)

2.3.2 - COMPLEXIDADE COMPUTACIONAL

No melhor caso (*Ômega* Ω), o cenário ideal ocorre quando a lista já está ordenada. Cada elemento é comparado apenas uma vez e não há trocas, resultando em um tempo de execução de $\Omega(n)$. No caso médio (*Theta* Θ), em média, cada inserção envolve a comparação com metade dos elementos já ordenados, levando a uma complexidade de $\Theta(n^2)$. No pior caso (*Big O*), quando a lista está em ordem reversa, o algoritmo realiza $n(n-1)/2$ comparações e trocas, resultando em uma complexidade de $O(n^2)$. (SONG, [s.d.])

2.3.3 - VANTAGENS E DESVANTAGENS

As principais vantagens do *Insertion Sort* incluem sua simplicidade de implementação, estabilidade, eficiência em listas pequenas ou quase ordenadas, baixo consumo de

memória e adaptabilidade, com o número de inversões sendo proporcional ao número de trocas, resultando em $O(n)$ em listas já classificadas. (GEEKSFORGEEKS, 2024)

As principais desvantagens incluem sua ineficiência em listas grandes e seu desempenho inferior em comparação com outros algoritmos de ordenação na maioria dos casos. (GEEKSFORGEEKS, 2024)

2.4 - BINARY INSERTION SORT

Binary Insertion Sort é semelhante ao *Insertion Sort*, mas não usa pesquisa linear, usa pesquisa binária. O *Binary Insertion Sort* é um algoritmo de organização que divide qualquer lista em duas partes, organizada e não organizada. Busca binária é um algoritmo de busca que é usado para encontrar a posição de um valor dentro de um conjunto organizado, funciona dividindo repetidamente a busca pela metade até o valor ser encontrado, como podemos observar na Figura 4. (GEEKSFORGEEKS, 2023)

Figura 4 - Busca binária



Fonte: GeekForGeeks (2024)

2.4.1 - FUNCIONAMENTO

O *Binary Insertion Sort* é um algoritmo de ordenação que pega um conjunto de dados e alinha em duas partes, organizada e não organizada. O algoritmo seleciona um

elemento de cada subconjunto de cada vez e alinha na posição correta. O alinhamento é realizado da esquerda para a direita. (GEEKSFORGEEEKS, 2023)

2.4.2 - COMPLEXIDADE COMPUTACIONAL

No melhor caso (Ω), onde o conjunto já está ordenado, a busca binária encontra rapidamente a posição para inserir o novo elemento, resultando em um tempo de execução de $\Omega(n)$. No caso médio (Θ), em média, cada inserção requer uma busca binária seguida de uma inserção, resultando em um tempo de execução de $\Theta(n \log n)$. No pior caso (O), ocorre quando a lista está em ordem reversa, levando o algoritmo a fazer o máximo de inserções e comparações, portanto, a complexidade é $O(n^2)$. (SONG, [s.d.])

2.4.3 - VANTAGENS E DESVANTAGENS

As principais vantagens desse algoritmo incluem a sua eficácia em organizar conjuntos curtos e semi-organizados, a rapidez ao lidar com essas listas e a simplicidade tanto na compreensão quanto na implementação. No entanto, suas desvantagens são notáveis quando se trata de listas grandes, onde sua performance tende a ser significativamente mais lenta. (GEEKSFORGEEEKS, 2023)

2.5 - MERGE SORT

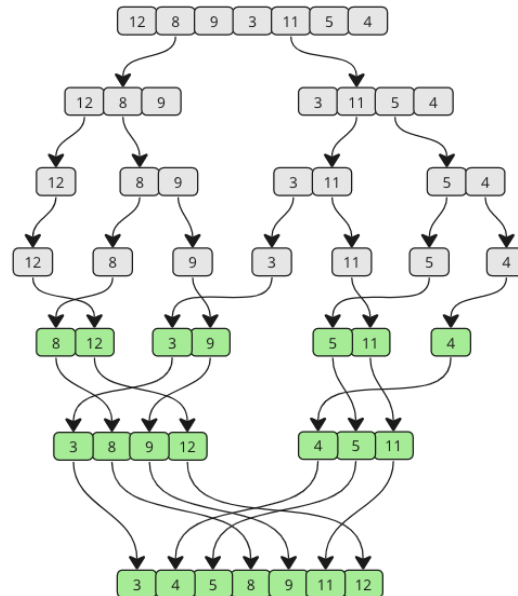
O algoritmo *Merge Sort* é um algoritmo de ordenação baseado na técnica de dividir para conquistar. Dividimos um conjunto, originalmente não ordenado, em subconjuntos menores, até que se torne atômico, ou seja, não divisível. Após a divisão dos vetores, tais divisões são combinadas já realizando o processo de verificação entre os números de cada conjunto que está sendo unificado, para enfim formar uma lista ordenada corretamente com todos os elementos. (W3SCHOOLS, 2024)

2.5.1 - FUNCIONAMENTO

Para ordenar o seguinte *array* [5, 3, 8, 6]: Realizamos a divisão sempre pela metade, resultando em [5, 3] e [8, 6], que ao dividir novamente, temos: [5] [3] e [8] [6], em seguida realizamos o processo de junção das sublistas, comparando sempre cada elemento da sublista, com o correspondente da outra [3, 5] e [6, 8] → [3, 5, 6, 8]. Em seguida, é realizada a junção das sublistas, comparando sempre cada elemento da

sublista, com o correspondente da outra sublista conforme mostra a Figura 5. (W3SCHOOLS, 2024)

Figura 5 - Exemplo de Ordenação com Merge Sort



Fonte: W3Schools (2024)

2.5.2 - COMPLEXIDADE COMPUTACIONAL

No melhor caso (Ω), o *Merge Sort* sempre divide a lista e combina os elementos independentemente da ordem. Portanto, mesmo no melhor cenário, a complexidade é $\Omega(n \log n)$. No caso médio (Θ), a complexidade permanece $\Theta(n \log n)$, já que o algoritmo continua a dividir e mesclar os elementos. No pior caso (O), a complexidade também é $O(n \log n)$, pois o *Merge Sort* segue consistentemente o mesmo padrão de divisão e combinação, independentemente da ordem inicial dos elementos (SONG, [s.d.]).

2.5.3 - VANTAGENS E DESVANTAGENS

As principais vantagens desse algoritmo são, a constância do algoritmo quando se trata do seu grau de complexidade que nos garante que será previsível o desempenho, tanto no melhor, quanto no pior caso, podemos destacar também a facilidade de sua implementação, onde, por ser um algoritmo que utiliza da técnica “dividir para conquistar”, sua abordagem é mais direta, sendo mais fácil de implementá-lo em qualquer aplicação sem muitas dificuldades e além do fato de que

a junção das sublistas serem realizadas individualmente, permite que seja implementado um processamento paralelo de forma “natural”, aprimorando o desempenho da aplicação. (GEEKSFORGEEKS, 2013)

Apesar de tantas vantagens, o algoritmo *Merge Sort* apresenta também diversas desvantagens. Sendo uma de suas principais a sua complexidade de espaço onde, devido a criação das sub listas para o processo de divisão da lista originalmente não ordenada, há a necessidade de maior capacidade de armazenamento na memória, outra desvantagem do algoritmo é quando se comparado com outros algoritmos que utilizam da mesma técnica, o mesmo pode sair em desvantagem quanto ao consumo de memória e uso de *Cache*, como por exemplo o algoritmo *Quick Sort*, que tende a ser mais “amigável” nesses casos quanto a esse consumo. (GEEKSFORGEEKS, 2013)

3 - DESENVOLVIMENTO

Para o desenvolvimento do nosso trabalho começamos com uma função base, vista na Figura 6, para leitura dos arquivos disponibilizados. Essa função é responsável por ler os números no arquivo de texto(.txt), convertê-los em tipo inteiro(int) e inserir na lista LinhasArquivo.

Figura 6 - Função Entrada()

APS - EntradaDados.py

```
1 def Entrada():
2     CaminhoArquivo = r'C:\wamp64\www\APS\10000_numbers.txt'
3     LinhasArquivo = []
4     with open(CaminhoArquivo, 'r', encoding='utf-8') as ArquivoTxt:
5         for Linha in ArquivoTxt:
6             LinhasArquivo.append(int(Linha.strip()))
7     return LinhasArquivo
```

Fonte: Autoria Própria (2024)

Para obtermos a média de tempo sobre cada algoritmo, utilizamos a biblioteca *timeit* onde elaboramos a função *MedicaoTempo()*, essa função tem o objetivo de receber os parâmetros que foram definidos na variável *TempoExecucao*, que define quantas vezes o algoritmo selecionado será executado e logo em seguida o tempo de execução é exibido, conforme evidenciado na Figura 7.

Figura 7 - Função MedicaoTempo()

APS - MergeSort.py

```
1 def MedicaoTempo():
2     ArrayMS = Entrada()
3     TempoExecucao = timeit.timeit(lambda: MergeSort(ArrayMS), number = 10)
4     print(TempoExecucao)
5
6 MedicaoTempo()
```

Fonte: Autoria Própria (2024)

3.1 - BINARY INSERTION SORT

O *Binary Insertion Sort* é um algoritmo recursivo, separado em 3 funções. Sendo elas a função `BinaryInsertionSort()` que recebe como entrada a variável *ArrayBIS*, dentro da função existe um laço, uma variável *Valor* que armazena o valor do elemento atual da lista na posição informada pelo laço de repetição através da variável *i*, depois a variável auxiliar recebe o resultado e inicia a função `BuscaBinaria()`. Por último a variável *ArrayBIS* novamente é utilizada para ajustar e inserir as informações recebidas pela variável *Valor* na posição auxiliar através do fatiamento de lista, mantendo a parte da esquerda ordenada. Ao final retorna novamente a variável fornecida inicialmente a *ArrayBIS*, abaixo podemos ver a função ilustrada pela Figura 8. (GEEKSFORGEEEKS, 2023)

Figura 8 - Função Binary Insertion Sort

```
21 def BinaryInsertionSort(ArrayBIS):
22     for i in range(1, len(ArrayBIS)):
23         Valor = ArrayBIS[i]
24         Auxiliar = BuscaBinaria(ArrayBIS, Valor, 0, i - 1)
25         ArrayBIS = ArrayBIS[:Auxiliar] + [Valor] + ArrayBIS[Auxiliar:i] + ArrayBIS[i + 1:]
26     return ArrayBIS
```

Fonte: Autoria Própria (2024)

A função `BuscaBinaria()` é chamada diversas vezes pela função anterior, essa função é iniciada recebendo as variáveis de entrada *ArrayBB*, *Valor*, *Início* e *Final*. Caso o *Início* seja igual o *final* e *ArrayBB* na posição dada por *Início* seja maior que o *Valor* da variável *Início* é retornado, caso contrário retorna *Início* somado a 1(um). Se *Início* for maior que *Final*, também retorna *Início*. Em sequência, a variável *Meio* recebe o quociente inteiro da divisão das variáveis *início* mais *final* divididas por dois, se *ArrayBB* na posição informada por *Meio* for menor que *Valor*, é retornado a função `BuscaBinaria()` com valores (*ArrayBB*, *Valor*, *Meio* somado a um e *Final*) respectivamente. Se *ArrayBB* na posição informada por *Meio* for maior que *Valor* então o retorno será a função `BuscaBinaria()` com valores (*ArrayBB*, *Valor*, *Início* e *Meio* subtraída por um), senão retorna apenas a variável *Meio*, abaixo na Figura 9 temos o código desenvolvido. (GEEKSFORGEEEKS, 2023)

Figura 9 - Função BuscaBinaria()

```
4 def BuscaBinaria(ArrayBB, Valor, Inicio, Final):
5     if Inicio == Final:
6         if ArrayBB[Inicio] > Valor:
7             return Inicio
8         else:
9             return Inicio + 1
10    if Inicio > Final:
11        return Inicio
12
13    Media = (Inicio + Final) // 2
14    if ArrayBB[Media] < Valor:
15        return BuscaBinaria(ArrayBB, Valor, Media + 1, Final)
16    elif ArrayBB[Media] > Valor:
17        return BuscaBinaria(ArrayBB, Valor, Inicio, Media - 1)
18    else:
19        return Media
```

Fonte: Autoria Própria (2024)

3.2 - INSERTION SORT

A função InsertionSort() quando iniciada recebe de entrada *Array/S*, em sequência a variável *QuantidadeLista* pega o tamanho de *Array/S* e um laço de repetição é iniciado com a quantidade de repetições definida pela informação de *QuantidadeLista*. A função itera sobre cada elemento da lista, começando do segundo elemento. Para cada elemento da lista, armazena-se o valor na variável *Numero* e define o índice *Auxiliar* como o índice anterior ao atual ($i - 1$). Em seguida, compara a variável *Numero* com os elementos ordenados da lista à esquerda. Enquanto a variável *Auxiliar* for maior ou igual a zero e a variável *Numero* for menor que o elemento no índice *Auxiliar* na lista *Array/S*, muda o elemento que está no índice *Auxiliar* uma posição à frente e o índice *Auxiliar* é diminuído em um. Após deslocar todos os elementos maiores que *Numero* para a direita, *Numero* é inserido na posição correta, a Figura 10 evidencia o código feito. (W3SCHOOLS, [s.d.])

Figura 10 - Função InsertionSort()

```
1 from EntradaDados import Entrada
2 import timeit
3
4 def InsertionSort(ArrayIS):
5     QuantidadeLista = len(ArrayIS)
6     for i in range(QuantidadeLista):
7         Numero = ArrayIS[i]
8         Auxiliar = i - 1
9         while Auxiliar >= 0 and Numero < ArrayIS[Auxiliar]:
10             ArrayIS[Auxiliar + 1] = ArrayIS[Auxiliar]
11             Auxiliar = Auxiliar - 1
12         ArrayIS[Auxiliar + 1] = Numero
```

Fonte: Autoria Própria (2024)

3.3 - MERGE SORT

A função MergeSort(), evidenciada na Figura 11, recebe a entrada *ArrayMS*, e para começar a implementação do algoritmo de ordenação, declara de três variáveis: 'Meio' para delimitar o meio da lista, dividindo seu tamanho por 2, 'Esquerda' e 'Direita' criando novas listas e dividindo a ordenada em partes. Após a divisão, o processo é repetido de forma recursiva nas metades, obtendo as menores partes de cada lista. Em seguida, são declaradas outras três variáveis: *IndiceEsquerda* que percorrerá a lista da esquerda, *IndiceDireita* para a lista da direita e *IndiceLista* para a lista principal, contendo os números ordenados com base nas iterações do código. Enquanto o valor do índice relacionado com sua respectiva lista for menor que o tamanho dessa lista, ainda há elementos a serem validados. O processo de verificação ocorre entre as metades esquerda e direita, comparando se o número do índice na lista da direita é maior ou menor que o valor correspondente na lista da esquerda. O menor elemento é adicionado à lista ordenada, e o índice dessa lista avança para o próximo item, enquanto o índice da outra lista permanece no mesmo valor para nova comparação, até que ele também seja maior e adicionado à lista dos números ordenados. Esse processo garante que os elementos estão corretamente ordenados. (NARASIMHA KARUMANCHI, 2017)

Figura 11 - Função MergeSort()

APS - MergeSort.py

```
1  from EntradaDados import Entrada
2  import timeit
3
4  def MergeSort(ArrayMS):
5      if len(ArrayMS) > 1:
6          Meio = len(ArrayMS) // 2
7          Esquerda = ArrayMS[:Meio]
8          Direita = ArrayMS[Meio:]
9
10         MergeSort(Esquerda)
11         MergeSort(Direita)
12
13         IndiceEsquerda = 0
14         IndiceDireita = 0
15         IndiceLista = 0
16
17         while IndiceEsquerda < len(Esquerda) and IndiceDireita < len(Direita):
18             if Esquerda[IndiceEsquerda] < Direita[IndiceDireita]:
19                 ArrayMS[IndiceLista] = Esquerda[IndiceEsquerda]
20                 IndiceEsquerda += 1
21             else:
22                 ArrayMS[IndiceLista] = Direita[IndiceDireita]
23                 IndiceDireita += 1
24             IndiceLista += 1
25
26         while IndiceEsquerda < len(Esquerda):
27             ArrayMS[IndiceLista] = Esquerda[IndiceEsquerda]
28             IndiceEsquerda += 1
29             IndiceLista += 1
30
31         while IndiceDireita < len(Direita):
32             ArrayMS[IndiceLista] = Direita[IndiceDireita]
33             IndiceDireita += 1
34             IndiceLista += 1
35
36
```

Fonte: Autoria Própria (2024)

5 - RESULTADOS E DISCUSSÃO

As análises dos algoritmos foram realizadas utilizando uma implementação na linguagem de programação *Python*. Para obter medições precisas dos tempos de execução, utilizamos a biblioteca *timeit* para executar a função de ordenação 10 vezes para cada conjunto de dados, calculando os tempos médios de execução. As execuções foram realizadas em uma máquina equipada com um processador Intel Core i7 de 7ª geração, 12 GB de RAM e um SSD de 480 GB, rodando o sistema operacional Windows 10.

5.1 - RESULTADOS DO MERGE SORT

Os resultados dos tempos de execução são apresentados na Tabela 1:

Tabela 1 - Resultados do *Merge Sort*

1000 Números	20ms
5000 Números	120,9ms
10000 Números	316,5ms

Fonte: Autoria Própria (2024)

A captura de tela das saídas do terminal mostra os tempos médios de execução para cada conjunto de dados, conforme evidenciado nas Figura 12.

Figura 12 - Saída do terminal *Merge Sort*

Media com 1000 números

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURACÃO  TERMINAL  PORTAS
[Running] python -u "c:\wamp64\www\APS\MergeSort.py"
0.020198999988699332
```

Media com 5000 números

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURACÃO  TERMINAL  PORTAS
[Running] python -u "c:\wamp64\www\APS\MergeSort.py"
0.12894189997878857
```

Media com 10000 números

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURACÃO  TERMINAL  PORTAS
[Running] python -u "c:\wamp64\www\APS\MergeSort.py"
0.3165887999930419
```

Fonte: Autoria Própria (2024)

Os resultados demonstram um crescimento quase linear dos tempos de execução em relação ao tamanho dos dados. Os resultados confirmam a eficiência do algoritmo em lidar com grandes volumes de dados, mantendo tempos de execução razoáveis mesmo para conjuntos maiores.

5.2 - RESULTADOS DO INSERTION SORT

Os resultados dos tempos de execução são apresentados na Tabela 2:

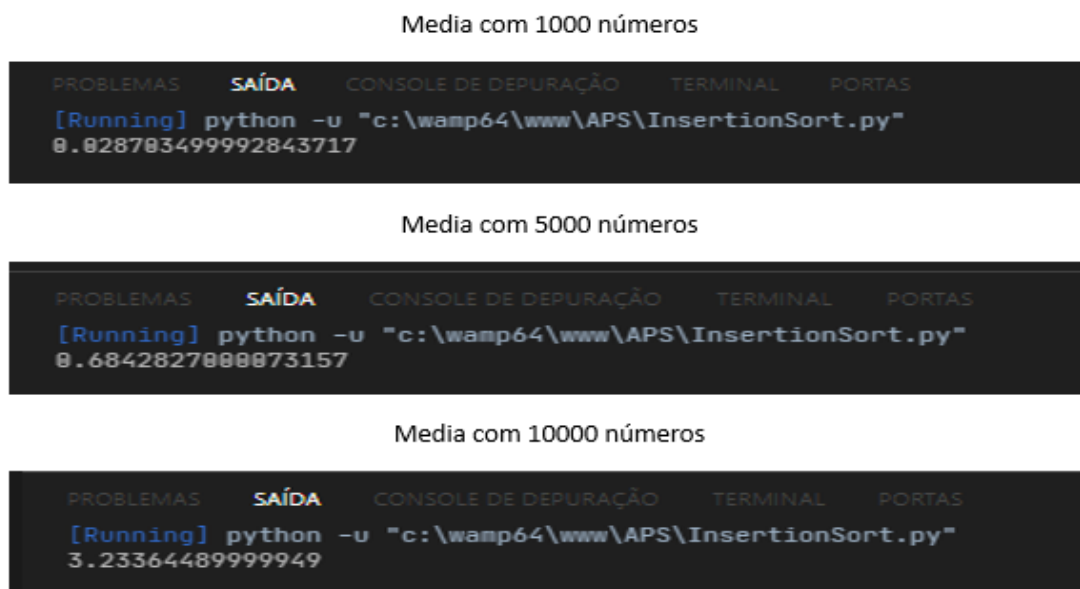
Tabela 2 - Resultados *Insertion Sort*

1000 Números	28,7ms
5000 Números	684,2ms
10000 Números	3233,6ms

Fonte: Autoria Própria (2024)

As capturas de tela das saídas do terminal mostram os tempos médios de execução para cada conjunto de dados, conforme evidenciado na Figura 13.

Figura 13 - Saída do terminal Insertion Sort



Fonte: Autoria Própria (2024)

Os resultados demonstram que o tempo de execução do algoritmo Insertion Sort cresce exponencialmente com o aumento do tamanho do conjunto de dados. Podemos dizer que o *Insertion Sort* seja eficiente para conjuntos de dados pequenos, mas seu desempenho se cai rapidamente à medida que o tamanho dos dados aumenta.

5.3 - RESULTADOS DO BINARY INSERTION SORT

Os resultados dos tempos de execução são apresentados na Tabela 3:

Tabela 3 - Resultados *Binary Insertion Sort*

1000 Números	134,8ms
5000 Números	2681,6ms
10000 Números	13016,7ms

Fonte: Autoria Própria (2024)

As capturas de tela das saídas do terminal mostram os tempos médios de execução para cada conjunto de dados, conforme evidenciado na Figura 14.

Figura 14 - Saída do terminal *Binary Insertion Sort*

Media com 1000 numeros

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
[Running] python -u "c:\wamp64\www\APS\BinaryInsertionSort.py"
0.13483279998763464
```

Media com 5000 numeros

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
[Running] python -u "c:\wamp64\www\APS\BinaryInsertionSort.py"
2.6816684000077657
```

Media com 10000 numeros

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
[Running] python -u "c:\wamp64\www\APS\BinaryInsertionSort.py"
13.016752800001996
```

Fonte: Autoria Própria (2024)

Os resultados demonstram que o tempo de execução do *Binary Insertion Sort* cresce significativamente com o aumento dos dados. Podemos observar que embora a busca binária possa reduzir o número de comparações, o tempo de execução ainda cresce de forma exponencial em relação ao tamanho dos dados. Esses resultados confirmam a ineficiência do algoritmo em lidar com grandes volumes de dados.

5.4 - COMPARAÇÃO DE RESULTADOS ENTRE ALGORITMOS

A Tabela 4 mostra os resultados obtidos durante os testes de cada algoritmo, evidenciando o desempenho e a escalabilidade dos diferentes algoritmos de ordenação. A comparação é relevante para entender a eficiência dos algoritmos testados.

Tabela 4 - Comparação entre os algoritmos

Algoritmo	1000 números (ms)	5000 números (ms)	10000 números (ms)
<i>Merge Sort</i>	20	120,9	316,5
<i>Insertion Sort</i>	28,7	684,2	3233,6
<i>Binary Insertion Sort</i>	134,8	2681,6	13016,7

Fonte: Autoria Própria (2024)

Os resultados evidenciam que o algoritmo *Merge Sort* é o mais eficiente entre os três, especialmente quando lidamos com conjuntos de dados maiores. O *Insertion Sort*, apesar de sua simplicidade, não apresentou bons resultados com os conjuntos de dados maiores. Por fim, o *Binary Insertion Sort* foi o menos eficiente dos três algoritmos para ordenar grandes volumes de informações.

7 - CONSIDERAÇÕES FINAIS

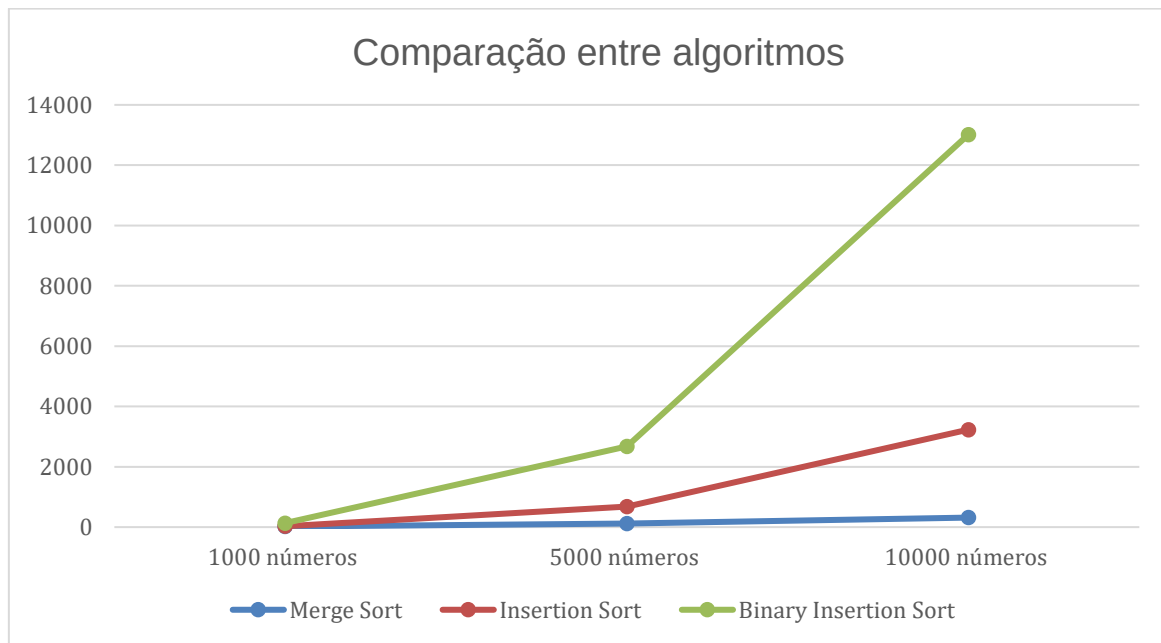
Os resultados obtidos a partir da análise de desempenho dos algoritmos *Merge Sort*, *Insertion Sort* e *Binary Insertion Sort* evidenciam diferenças significativas na eficiência e escalabilidade de cada método. Por meio dos testes que realizamos com três arquivos de números observamos que o *Merge Sort* se destacou como o algoritmo mais eficiente entre os três, especialmente quando aplicado a grandes volumes de dados.

O *Insertion Sort*, mesmo sendo um algoritmo simples e com uma implementação mais fácil, se mostrou não eficaz para conjuntos de dados maiores, apresentando um crescimento considerável no tempo de execução. Fazendo assim esse algoritmo mais adequado para conjuntos pequenos ou situações onde a simplicidade e a implementação rápida são mais importantes do que a eficiência.

Binary Insertion Sort, apesar de utilizar uma estratégia de busca binária para reduzir o número de comparações, não demonstrou um ganho significativo de desempenho em relação ao *Insertion Sort* tradicional. O crescimento exponencial do tempo de execução à medida que o tamanho dos dados aumentou evidenciou que ele não é nem um pouco prático para volume grande de informações.

Para exemplificarmos melhor, elaboramos o Gráfico 1 que mostra visualmente a perda de desempenho do *Binary Insertion sort* e evidencia o algoritmo *Merge Sort*.

Gráfico 1 - Comparação entre algoritmos



Fonte: Autoria Própria (2024)

Em conclusão, nossa análise apenas afirma a importância de selecionar o algoritmo de ordenação melhor apropriado a situação. O *Merge Sort* se apresenta como a melhor opção para grandes conjuntos de dados, enquanto o *Insertion Sort* e o *Binary Insertion Sort* podem ser considerados para casos específicos que envolvem conjuntos de dados menores.

REFERÊNCIAS BIBLIOGRÁFICAS

LOPES, R. **Algoritmo: o que é e como funciona em programação e rede social.** Disponível em: <<https://hub.asimov.academy/blog/o-que-sao-algoritmos/>>. Acesso em: 10 nov. 2024.

GEEKSFORGEES. **Insertion Sort Algorithm.** Disponível em: <<https://www.geeksforgeeks.org/insertion-sort-algorithm/>>. Acesso em: 6 nov. 2024.

W3SCHOOLS. **DSA Merge Sort.** Disponível em: <https://www.w3schools.com/dsa/dsa_algo_mergesort.php>. Acesso em: 6 nov. 2024.

RIZZON, J. O **que é: Binary Sort.** Disponível em: <<https://napoleon.com.br/glossario/o-que-e-binary-sort/>>. Acesso em: 10 nov. 2024.

SEDGEWICK, R.; WAYNE, K. **Algorithms.** Boston: Addison-Wesley, 2016.

CORMEN, T. H.; AL, E. Algoritmos: teoria e prática. Rio De Janeiro: Campus, 2012.

RIBEIRO, A. **Complexidade de algoritmos: uma questão de tempo e espaço.** Disponível em: <<https://pt.linkedin.com/pulse/complexidade-de-algoritmos-uma-quest%C3%A3o-tempo-e-espa%C3%A7o-ai-rel-ribeiro>>. Acesso em: 29 out. 2024.

BRUNET, J. A. **Análise Assintótica.** Disponível em: <<https://joaoarthurbm.github.io/eda/posts/analise-assintotica/>>. Acesso em: 29 out. 2024.

SANCHO, F. **Análise de algoritmo e análise assintótica: um resumo.** Disponível em: <<https://www.dio.me/articles/analise-de-algoritmo-e-analise-assintotica-um-resumo>>. Acesso em: 29 out. 2024.

Fernando Araujo. **Análise de algoritmo e análise assintótica: um resumo.** Disponível em: <<https://www.dio.me/articles/analise-de-algoritmo-e-analise-assintotica-um-resumo>>. Acesso em: 29 out. 2024.

SONG, S. W. Complexidade de Algoritmos. Disponível em: <<https://www.ime.usp.br/~song/mac5710/slides/01complex.pdf>>. Acesso em: 29 out. 2024.

KIELKOWSKI, M. **O que é big O Notation?** Disponível em: <<https://medium.com/linkapi-solutions/o-que-%C3%A9-big-o-notation-32f171e4a045>>. Acesso em: 28 out. 2024.

GEEKSFORGEEKS. **Insertion Sort Algorithm.** Disponível em: <<https://www.geeksforgeeks.org/insertion-sort-algorithm/>>. Acesso em: 31 out. 2024.

BRUNET, J. A. **Ordenação por Comparação: Insertion Sort.** Disponível em: <<https://joaoarthurbm.github.io/eda/posts/insertion-sort/>>. Acesso em: 6 nov. 2024.

GEEKSFORGEEKS. **Time and Space Complexity Analysis of Binary Search Algorithm.** Disponível em: <<https://www.geeksforgeeks.org/complexity-analysis-of-binary-search/>>. Acesso em: 31 out. 2024.

GEEKSFORGEEKS. **Binary Insertion Sort.** Disponível em: <<https://www.geeksforgeeks.org/binary-insertion-sort/>>. Acesso em: 31 out. 2024.

GEEKSFORGEEKS. **Merge Sort – Data Structure and Algorithms Tutorials.** Disponível em: <<https://www.geeksforgeeks.org/merge-sort/#applications-of-merge-sort>>. Acesso em: 6 nov. 2024.

W3SCHOOLS. **DSA Insertion Sort.** Disponível em: <https://www.w3schools.com/dsa/dsa_algo_insertionsort.php>. Acesso em: 9 nov. 2024.

NARASIMHA KARUMANCHI. **Data structures and algorithmic thinking with Python.** Hyderabad, India: Careermonk Publications, 2017.



RA: G783HF2

TURNNO: NOTURNO

SEMESTRE: 4º SEMESTRE

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

DATA: / /

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO



RA: N079657

TURNO: NOTURNO

SEMESTRE: 4º SEMESTRE

[illegible]

(1) Horas atribuídas de acordo com o regulamento das Atividades Supervisionadas do curso.

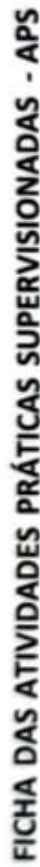
TOTAL DE HORAS ATRIBUÍDAS:

AVALIAÇÃO: _____

NOTA: _____

DATA: / /

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO



RA: N239239

TURNNO: NOTURNO

SEMESTRE: 4º SEMESTRE

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS:

Aprovado ou Reprovado

DATA: ____/____/____

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO



RA: T296DC0

TURNNO: NOTURNO

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

DATA: / /

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

RA: N296445

TURNNO: NOTURNO

SEMESTRE: 4º SEMESTRE

(1) Horas atribuídas de acordo com o regulamento das Atividades Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS:

AValiação: _____

NOTA: _____

DATA: / /

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO



RA: N2409H8

TURNNO: NOTURNO

SEMESTRE: 4º SEMESTRE

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS:

AVALIAÇÃO:

Aprovado ou Reprovado

NOTA:

DATA: / /

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

[illegible]

(11) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS:

AVALIAÇÃO:

Aprovado ou Reprovado

NOTA:

DATA: / /

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO